

[Edit](#)**Guided**

Automate CI Workflows by Using GitHub Actions

Challenge Overview

Understand the scenario

You are a DevOps Engineer at Hexelo, a company that wants to standardize and accelerate its continuous integration (CI) processes. In this Challenge Lab, you will create a GitHub repository and use GitHub Copilot in Visual Studio Code (VS Code) to generate an initial CI workflow for a Python project. Next, you will enhance the workflow by adding a linting job and establishing job dependencies. Finally, you will optimize the CI process by implementing dependency caching and a build matrix to test against multiple Python versions.

 You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

Navigating the Challenge Lab

Unable to process content from <https://raw.githubusercontent.com/LODSContent/Challenge-V3-Framework/main/Templates/Environments/None.md>

 ► Quick tips for navigating the Challenge Lab instructions.

[New to Challenge Labs? Click here to learn more.](#)

Create a GitHub repository and initial CI workflow

Hints Enabled

No Yes

 You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

- Sign in to the virtual machine as **LabUser** by using **Passw0rd!** as the password.
- Enter your GitHub Username, Email Address, Password, and Account URL into the following associated textboxes:

GitHub Username

GitHub Email Address

GitHub Password

GitHub Account URL

- Open **Microsoft Edge**, browse to **https://github.com**, and then sign in as **<GithubUN>** by using **<GithubPW>** as the password.

 Expand this hint for guidance on signing in to GitHub.

- On the Windows taskbar, in Search, enter **Edge**, and then select **Microsoft Edge** to open a new browser.
- In the Address bar, enter  <https://github.com>, and then press **Enter**.
- On the GitHub home page, in the upper right-hand corner, select **Sign in**.
- On the *Sign in to GitHub* page, in Username or email address, sign in as  [<GithubUN>](#) using  [<GithubPW>](#) as the password.
- If presented with the *Save password* dialog, select **Never**.
- On the *Device verification* page, enter the **Device Verification Code** sent to the email address used to register the account.

- Create a new public GitHub repository named  [copilot-ci-lab](#).

 Expand this hint for guidance on creating a new public repository named copilot-ci-lab.

- On your GitHub account homepage, in the top-right corner, select the  **+** icon, and then select **New repository**.
- For **Repository name**, enter  [copilot-ci-lab](#).
- Ensure the repository is set to **Public**.
- Select the **Create repository** button.

 Want to learn more? Review the documentation on [creating a new repository](#). A unique repository name is required. Leave the **GitHub** browser window open for later steps in this lab.

- Sign in to  [Visual Studio Code](#) to use **Copilot** with your GitHub account as  [<GithubUN>](#) by using  [<GithubPW>](#) as the password.

💡 Expand this hint for guidance on signing in to GitHub in Visual Studio Code. ^

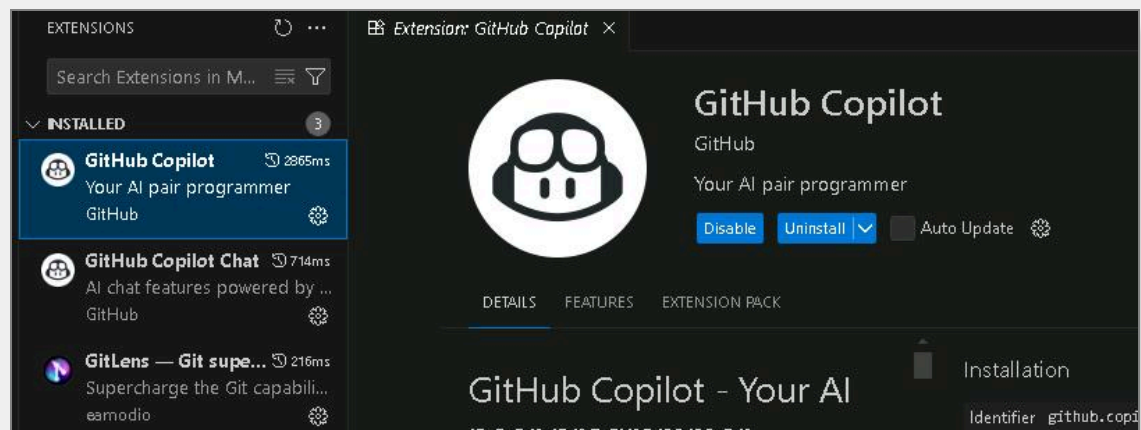
- On the Windows taskbar, in Search, enter **T** Visual Studio Code, and then select **Visual Studio Code**.
- In the bottom right-hand corner, select *Signed out*, select **Sign in to use Copilot**, and then sign in as **T** <GithubUN> by using **T** <GithubPW> as the password.
- Select **Continue**, and then select **Open** in the pop-up window.

📄 You may close the *Welcome* tabs.

- In **Visual Studio Code**, verify the GitHub Copilot extensions are installed.

💡 Expand this hint for guidance on verifying extensions. ^

- In **VS Code**, on the Activity Bar, select the **Extensions** icon.
- In the search box, enter **T** GitHub Copilot, and then verify the **GitHub Copilot** extension is installed.

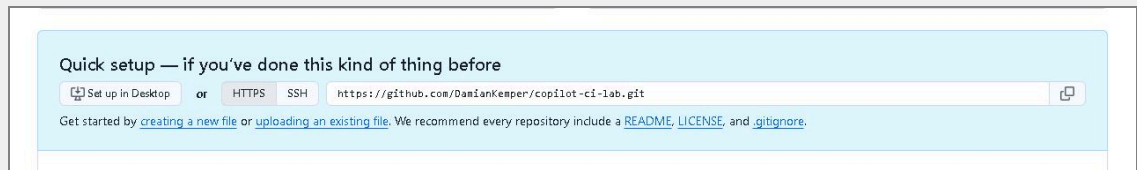


- In the search box, enter **T** GitHub Copilot Chat, and then verify the **GitHub Copilot Chat** extension is installed.
- If prompted, authorize GitHub Copilot and Copilot Chat by signing in to GitHub and completing the authorization steps.

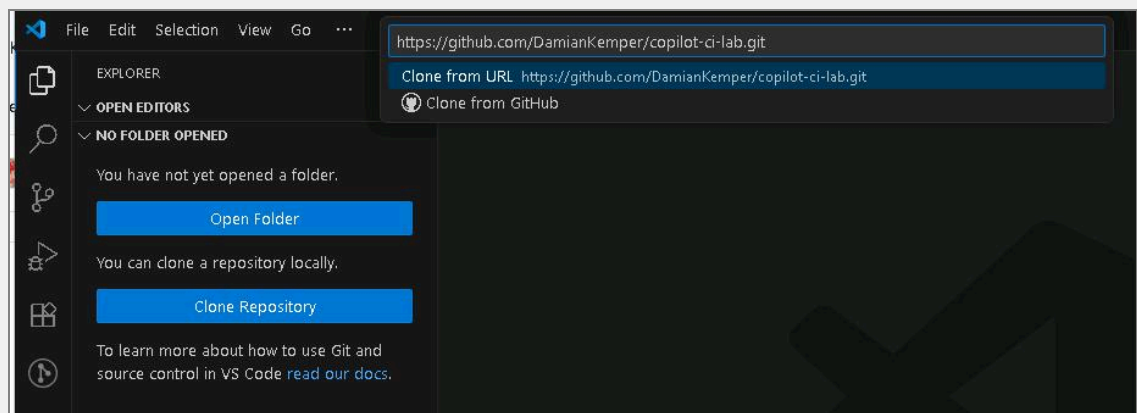
- Clone the **copilot-ci-lab** repository to the **T** D:\LabFiles folder.

💡 Expand this hint for guidance on cloning a project from GitHub.

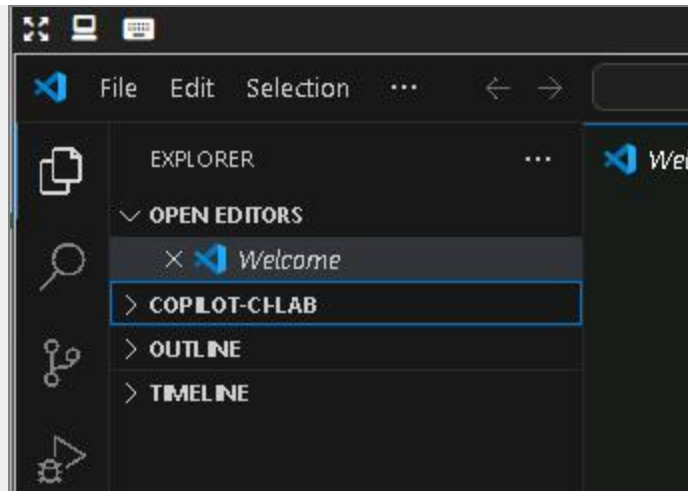
- In your browser, on the **copilot-ci-lab** repository page, select the **Copy** button next to the **HTTPS** clone URL.



- In **VS Code**, in the *Explorer* panel, select **Clone Repository**.
- Paste the repository URL you copied, and then select **Clone from URL**.



- If necessary, on the *The extension 'GitHub' wants to sign in using GitHub* pop-up, select **Allow**.
- If necessary, on the *additional permissions* page, select **Authorize Visual-Studio-Code**, and then select **Open**.
- When prompted, choose the **T D:\LabFiles** folder.
- When prompted **Would you like to open the cloned repository?**, select **Open** to open the **copilot-ci-lab** folder in **VS Code**.



- On the *Do you trust the authors of the files in this folder?* dialog, select **Yes, I trust the authors**.



Want to learn more? Review the documentation on [opening a folder in VS Code](#).

- Create the `app.py` Python application file in the **COPILLOT-CI-LAB** folder by using the following code:

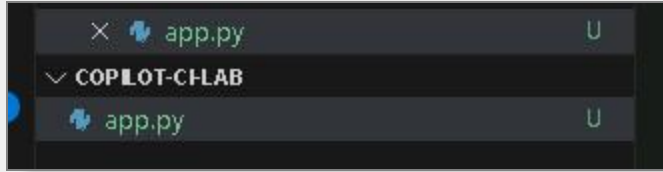
Python



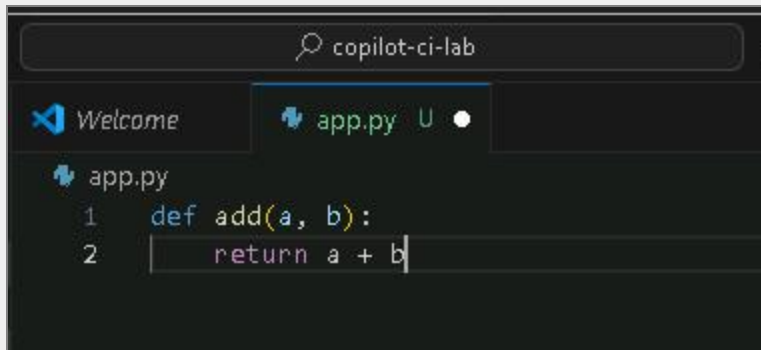
```
def add(a, b):  
    return a + b
```

💡 Expand this hint for guidance on creating the `app.py` Python application file. ^

- In **VS Code**, in the **Explorer** view, ensure the **COPILOT-CI-LAB** folder is open.
- In the **COPILOT-CI-LAB** folder, select the **New File** icon, and then enter `T app.py`.



- In the **app.py** file, enter the Python code from above.



- Press **Ctrl+S** to save the file.

💡 Want to learn more? Review the documentation on [adding new files](#).

- Create the `T test_app.py` pytest test file in the **COPILOT-CI-LAB** folder by using the following code:

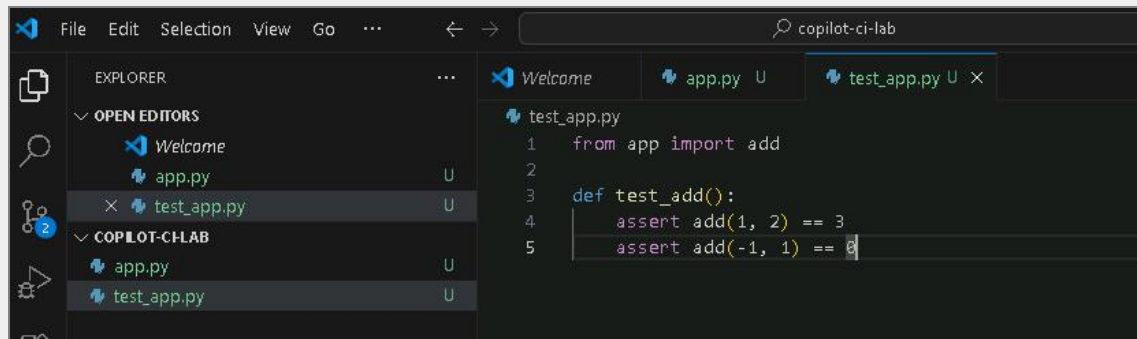
Python

```
from app import add

def test_add():
    assert add(2, 3) == 5
```

💡 Expand this hint for guidance on creating the `test_app.py` file. ^

- In the **Explorer** view in **VS Code**, in the **COPILOT-CI-LAB** folder, select the **New File** icon, and then enter `T` `test_app.py`.
- In the `test_app.py` file, enter the Python code from above.



- Press **Ctrl+S** to save the file.

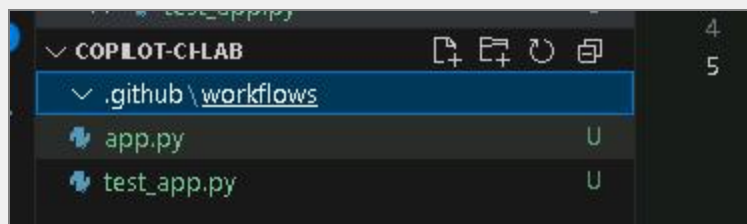
💡 Want to learn more? Review the documentation on [getting started with pytest](#).

❓ These simple files provide a basis for the CI workflow to run against. The application has a function, and the test file verifies its behavior.

- In the **COPILOT-CI-LAB** folder, create a new folder named `T` `.github`, and then create a subfolder named `T` `workflows`.

💡 Expand this hint for guidance on creating folders in VS Code. ^

- In the **Explorer** view in **VS Code**, in the **COPILOT-CI-LAB** folder, select the **New Folder** icon, and then enter `T` `.github`.
- In the `.github` folder, select the **New Folder** icon, and then enter `T` `workflows`.



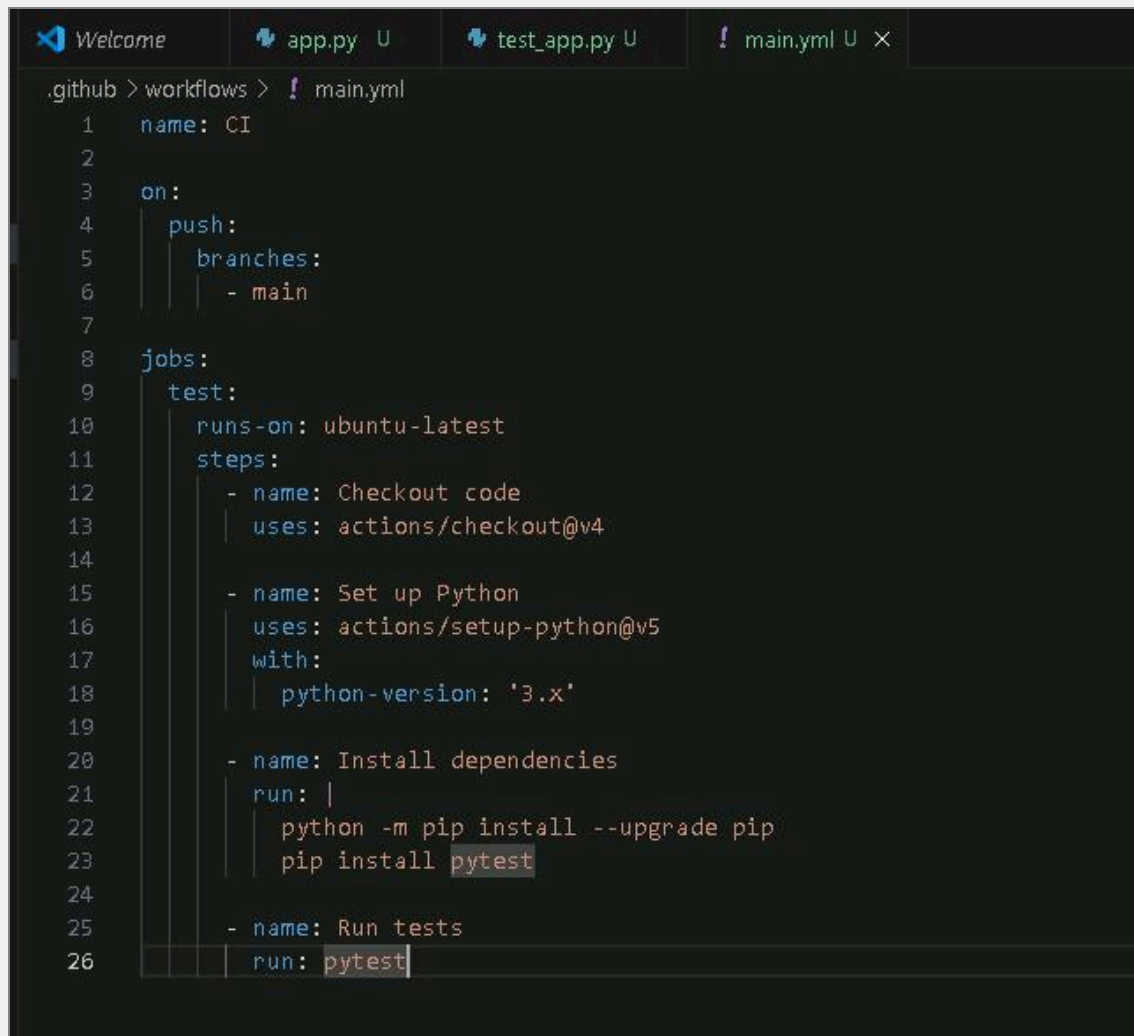
- Create a new file named `T` `main.yml` in the `workflows` folder, and then use GitHub Copilot in **VS Code** to generate a basic CI workflow that runs when changes are pushed to the **main** branch, by using the following prompt:



Create a GitHub Actions workflow for this repository in `.github/workflows/main.yml` that runs on push to the main branch and includes a single job called "test" that uses ubuntu-latest, checks out the code, sets up Python 3.x, installs pytest, and runs pytest.

💡 Expand this hint for guidance on generating a basic CI workflow triggered on push to the main branch. ^

- In the **workflows** folder, select the **New File** icon, and then enter **T** `main.yml`.
- Press **Ctrl+I** to open the **GitHub Copilot Inline Chat** textbox, enter the prompt from above, and then select **Send**.
- Review the YAML workflow that Copilot generates.
- If the response looks correct, select **Accept** to use the suggested code in the **main.yml** file.



```
.github > workflows > ! main.yml
1  name: CI
2
3  on:
4    push:
5      branches:
6        - main
7
8  jobs:
9    test:
10     runs-on: ubuntu-latest
11     steps:
12       - name: Checkout code
13         uses: actions/checkout@v4
14
15       - name: Set up Python
16         uses: actions/setup-python@v5
17         with:
18           python-version: '3.x'
19
20       - name: Install dependencies
21         run: |
22           python -m pip install --upgrade pip
23           pip install pytest
24
25       - name: Run tests
26         run: pytest
```

- Press **Ctrl+S** to save the **main.yml** file.

💡 Want to learn more? Review the documentation on [events that trigger workflows](#).

📄 GitHub Copilot suggestions can vary. ^

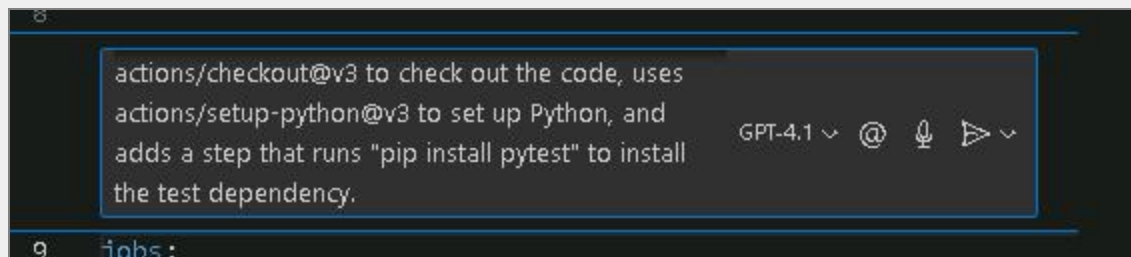
If the initial suggestion is not what you expect, you can ask Copilot Chat to regenerate the answer, refine your prompt, or manually edit the YAML directly in **VS Code**.

- Refine the workflow in the **main.yml** file so that the **test** job checks out the code, sets up Python, installs dependencies, and is ready to run *pytest* by using the following prompt:

Update `.github/workflows/main.yml` so that the "test" job runs on `ubuntu-latest`, uses `actions/checkout@v3` to check out the code, uses `actions/setup-python@v3` to set up Python, and adds a step that runs `"pip install pytest"` to install the test dependency.

💡 Expand this hint for guidance on refining the test job in the workflow file. ^

- In **VS Code**, ensure the **main.yml** file in the **.github/workflows** folder is open.
- In the **main.yml** file, select the entire script, and then in the **GitHub Copilot Inline Chat**, enter the prompt from above.



- Review Copilot's proposed changes to the workflow. If the changes look correct, apply them to your **main.yml** file by selecting **Accept**.
- Confirm that your workflow now contains a **test** job with steps that:
 - Checkout code.
 - Set up Python.
 - Install **pytest**.

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3
```

- Press **Ctrl+S** to save the file.

💡 Want to learn more? Review the documentation on [using GitHub Copilot Chat in your IDE](#).

- If necessary, add the following step to run *pytest*:

yaml



```
- name: Run tests
  run: pytest
```

💡 Expand this hint for guidance on adding a step to run *pytest*. ^

- In the **main.yml** editor in **VS Code**, locate the **test** job.
- If it does not already exist, add the code from above manually or by using Copilot, at the end of the job's steps.

```
.github > workflows > ! main.yml
9   jobs:
10    test:
11      steps:
12        - name: Install pytest
13          run: pip install pytest
14
15      runs-on: ubuntu-latest
16      steps:
17        - name: Checkout code
18          uses: actions/checkout@v3
19
20        - name: Set up Python
21          uses: actions/setup-python@v3
22          with:
23            python-version: '3.x'
24
25        - name: Install pytest
26          run: pip install pytest
27
28        - name: Run tests
29          run: pytest
```

- Press **Ctrl+S** to save the file.

- Set your Git Identity by using the `git config` command and using `<Name>` as the **username** and `<GithubUN>` as the **email address**.

💡 Expand this hint for guidance on configuring basic Git settings, such as username and email. ^

- In **VS Code**, select **Terminal > New Terminal** to open the integrated terminal.
- Enter `git config --global user.name "<Name>"` in the terminal to set your global Git username.
- Enter `git config --global user.email "<GithubUN>"` to set your global Git email address.

📄 Use `git config --list` to verify your configurations.

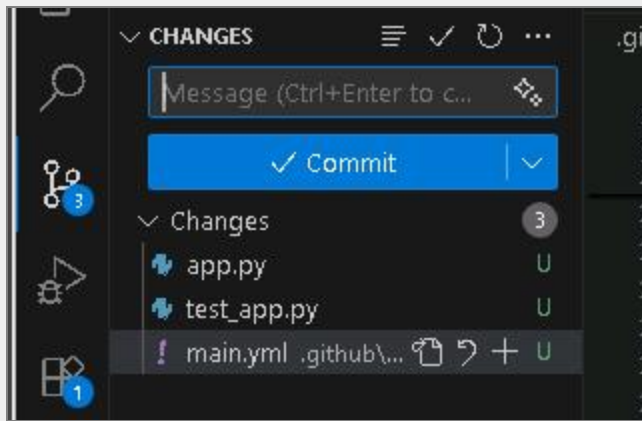
```
PS C:\Users\LabUser> git config --global user.name
PS C:\Users\LabUser> git config --global user.email
PS C:\Users\LabUser> git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=damian
user.email=damian.kemper@skillable.com
PS C:\Users\LabUser>
```

💡 Want to learn more? Review the documentation on [Git configuration](#).

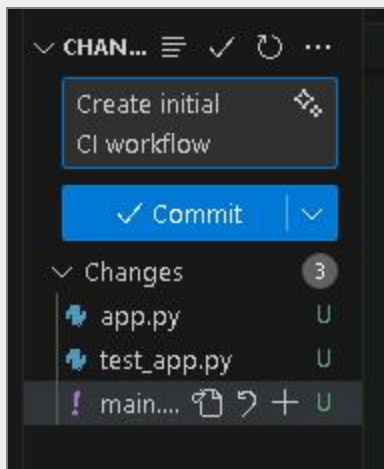
- Commit the **main.yml** workflow file by using **VS Code** and `Create initial CI workflow` as a commit message.

💡 Expand this hint for guidance on committing the main.yml workflow file.

- In the Activity Bar, select the **Source Control** icon.
- In the *Source Control* view, verify that **.github/workflows/main.yml** is listed as a changed file.



- In the **Message** box, enter **Create initial CI workflow**.



- Select **Commit** to commit your changes, and then select **Publish Branch**.
- On the *There are no staged changes to commit* dialog, select **Yes**.

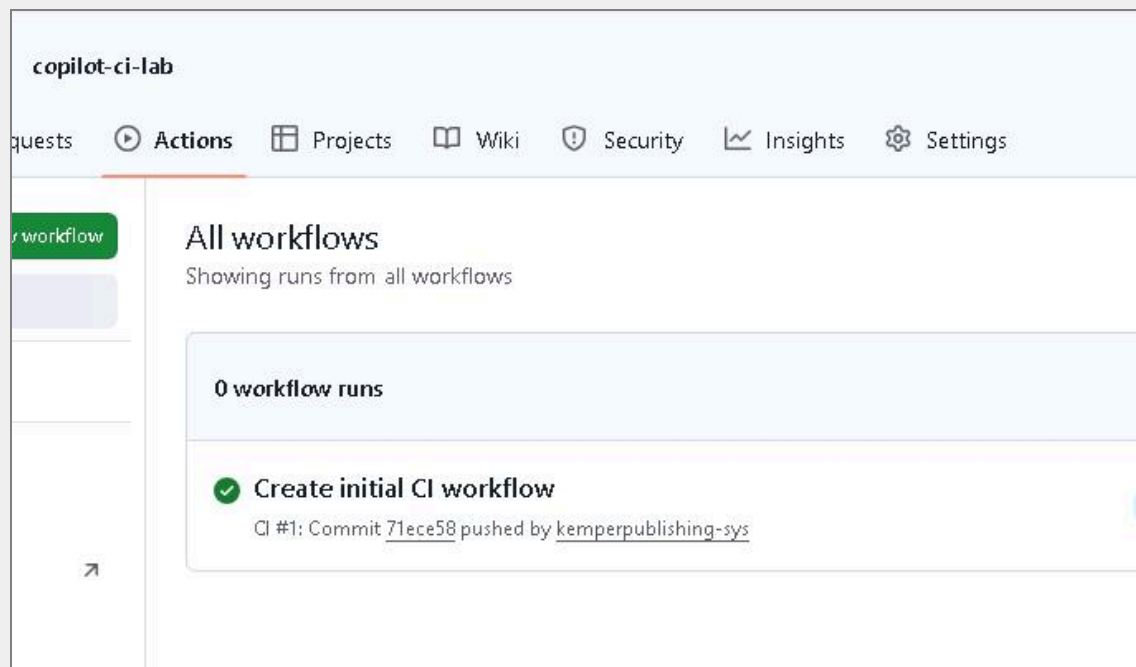
💡 Want to learn more? Review the documentation on [creating commits](#).

📄 If you selected **Commit** only, select the **Publish Branch** (or **Sync Changes / Push**) button to push your changes to GitHub, and then select **Authorize git-ecosystem**.

⚠️ Pushing your commit to the **main** branch will trigger the workflow immediately. This is expected for this lab.

- On the *Connect to GitHub* dialog, sign in with your browser as T <GithubUN> by using T <GithubPW> as the password.

? You can view the workflow from the Actions tab of your GitHub repository.



Check your work

Verify

Enhance the CI workflow with a linting job

Hints Enabled

No Yes

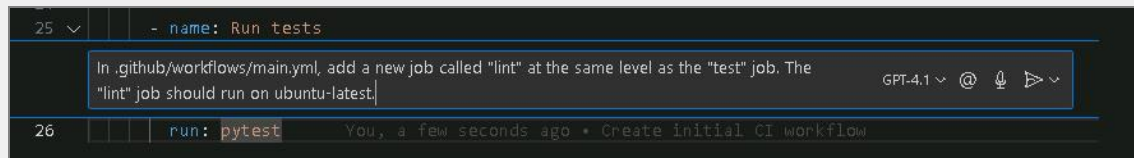
- Add a new job named `lint` to the **main.yml** workflow file by using the following:



In `.github/workflows/main.yml`, add a new job called "lint" at the same level as the "test" job. The "lint" job should run on ubuntu-latest.

💡 Expand this hint for guidance on adding a new job named **lint** to the workflow file. ^

- In **VS Code**, in the `.github/workflows/main.yml` file in the editor, select the entire script.
- Open the **GitHub Copilot Inline Chat** (**Ctrl+I**) and enter the above supplied prompt.



- Review the YAML Copilot proposes.
- Confirm that the **lint** job is at the same indentation level as the **test** job and that it includes `runs-on: ubuntu-latest`.

```
lint:
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.x'

    - name: Install flake8
      run: |
        python -m pip install --upgrade pip
        pip install flake8

    - name: Run flake8
      run: flake8 .
```

- **Accept** the suggestion to the `main.yml` file and save your changes.

💡 Want to learn more? Review the documentation on [workflow syntax for GitHub Actions](#).

📄 Proper YAML indentation is critical for GitHub Actions workflows. Ensure the `lint` job is at the same level as the `test` job.

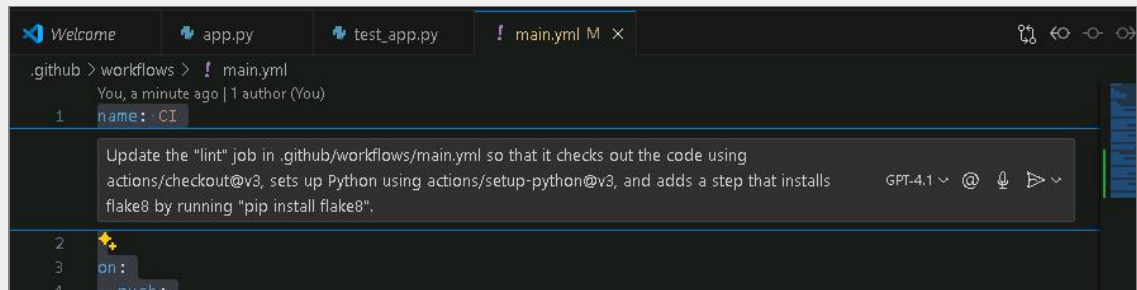
- Configure the `lint` job to install **flake8** by using the following:



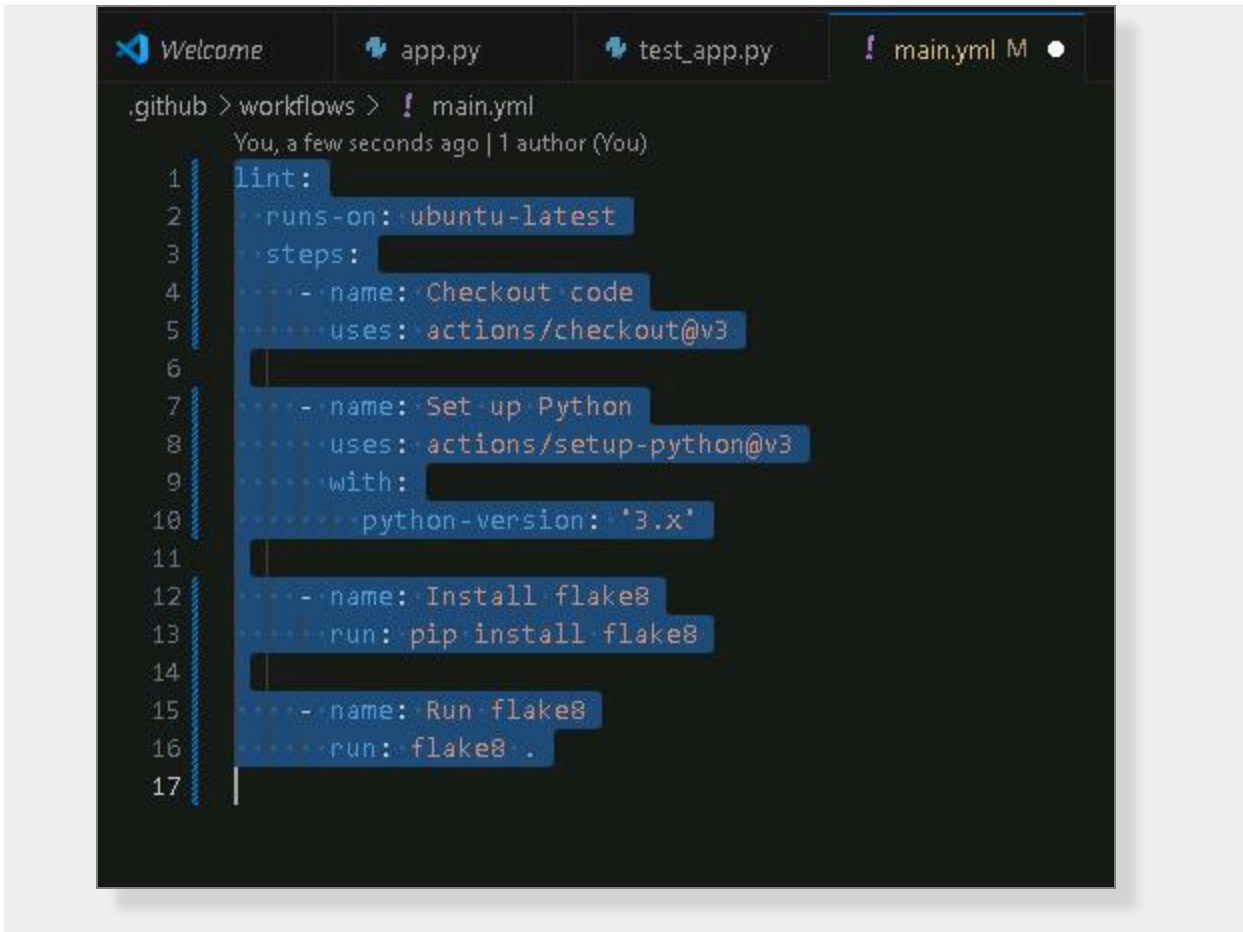
Update the "lint" job in `.github/workflows/main.yml` so that it checks out the code using `actions/checkout@v3`, sets up Python using `actions/setup-python@v3`, and adds a step that installs flake8 by running `"pip install flake8"`.

💡 Expand this hint for guidance on configuring the lint job to install **flake8**.

- In **VS Code**, in the **main.yml** editor, locate the **lint** job, and then highlight that section of the script.
- In **GitHub Copilot Inline Chat**, enter the above supplied prompt.



- Review the steps Copilot adds under the **lint** job and ensure they include:
 - A step using **actions/checkout@v3**.
 - A step using **actions/setup-python@v3**.
 - A step that runs **pip install flake8**.
 - Apply the updated YAML to the **main.yml** file and save your changes.



```
.github > workflows > ! main.yml
You, a few seconds ago | 1 author (You)
1  lint:
2    runs-on: ubuntu-latest
3    steps:
4      - name: Checkout code
5        uses: actions/checkout@v3
6
7      - name: Set up Python
8        uses: actions/setup-python@v3
9        with:
10         python-version: '3.x'
11
12      - name: Install flake8
13        run: pip install flake8
14
15      - name: Run flake8
16        run: flake8 .
17
```

 Want to learn more? Review the documentation on [flake8: Your Tool For Style Guide Enforcement](#).

 You may get no results for the prompt above if Copilot already generated the proper code in the first step.

- Add a step to the *lint* job to run **flake8** by using the following:

 In `.github/workflows/main.yml`, add a final step to the "lint" job that runs `flake8 .` to lint the repository.

💡 Expand this hint for guidance on adding a step to the lint job to run flake8.

- In **VS Code**, in the **main.yml** editor, highlight the **lint** job section of the script.
- In the **GitHub Copilot Inline Chat**, enter the above supplied prompt.

The screenshot shows the VS Code editor with the `main.yml` file open. The file content is as follows:

```

1 lint:
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

```

A GitHub Copilot inline chat prompt is visible, suggesting to add a final step to the "lint" job that runs "flake8 ." to lint the repository. The prompt text is: "In .github/workflows/main.yml, add a final step to the 'lint' job that runs 'flake8 .' to lint the repository." The prompt is attributed to GPT-4.1.

- Review the suggested step. It should look similar to:

yaml

```

- name: Run flake8
  run: flake8 .

```

The screenshot shows the VS Code editor with the `main.yml` file open. The file content is as follows:

```

1 - name: Run flake8
2   run: flake8 .
3

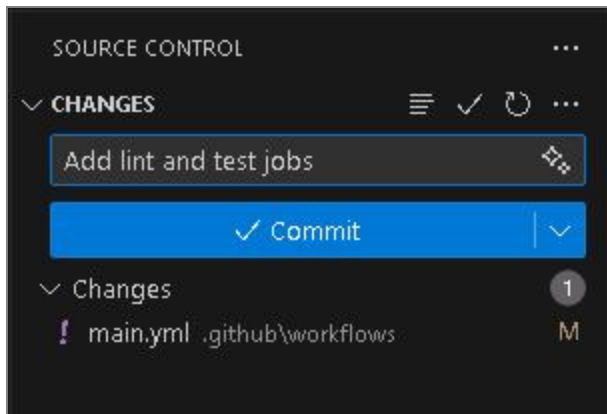
```

- Apply the change to **main.yml**, and verify that the step is part of the **lint** job. If it is not, manually add the **Run flake8** step above.
- Press **Ctrl+S** to save the file.



Want to learn more? Review the documentation on [using the run keyword](#).

- In the **Source Control** view in **VS Code**, confirm that `.github/workflows/main.yml` appears under **Changes**, and then enter the commit message `T Add lint and test jobs`.



- Select **Commit**, select **Yes**, and then select **Sync Changes** to push your changes to GitHub.
- Make the **test** job dependent on the **lint** job by entering `T needs: lint`.



Expand this hint for guidance on making the test job dependent on the lint job. ^

- In **VS Code**, in the `main.yml` file, locate the `test` job definition.
- Directly under the **test:** line, add `T needs: lint`.

Note: If the line `needs: lint` already exists under `runs-on:`, move it under the **test:** line.

- Ensure the indentation matches the other properties of the **test** job.
- Save the `main.yml` file.



Want to learn more? Review the documentation on [defining prerequisites for jobs](#).



The `needs` keyword creates a dependency graph. The `test` job will not start until the `lint` job has completed successfully.



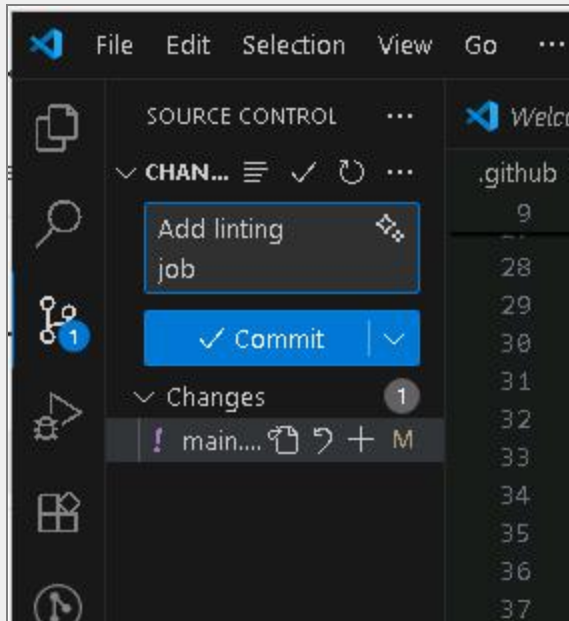
If the code has turned red, it is likely that the indentation is incorrect.

Open the **GitHub Copilot Inline Chat**, and enter the prompt: `T Fix the indentation..`

- Commit the changes by using `T Add linting job` as the **Commit** message.

💡 Expand this hint for guidance on committing the changes.

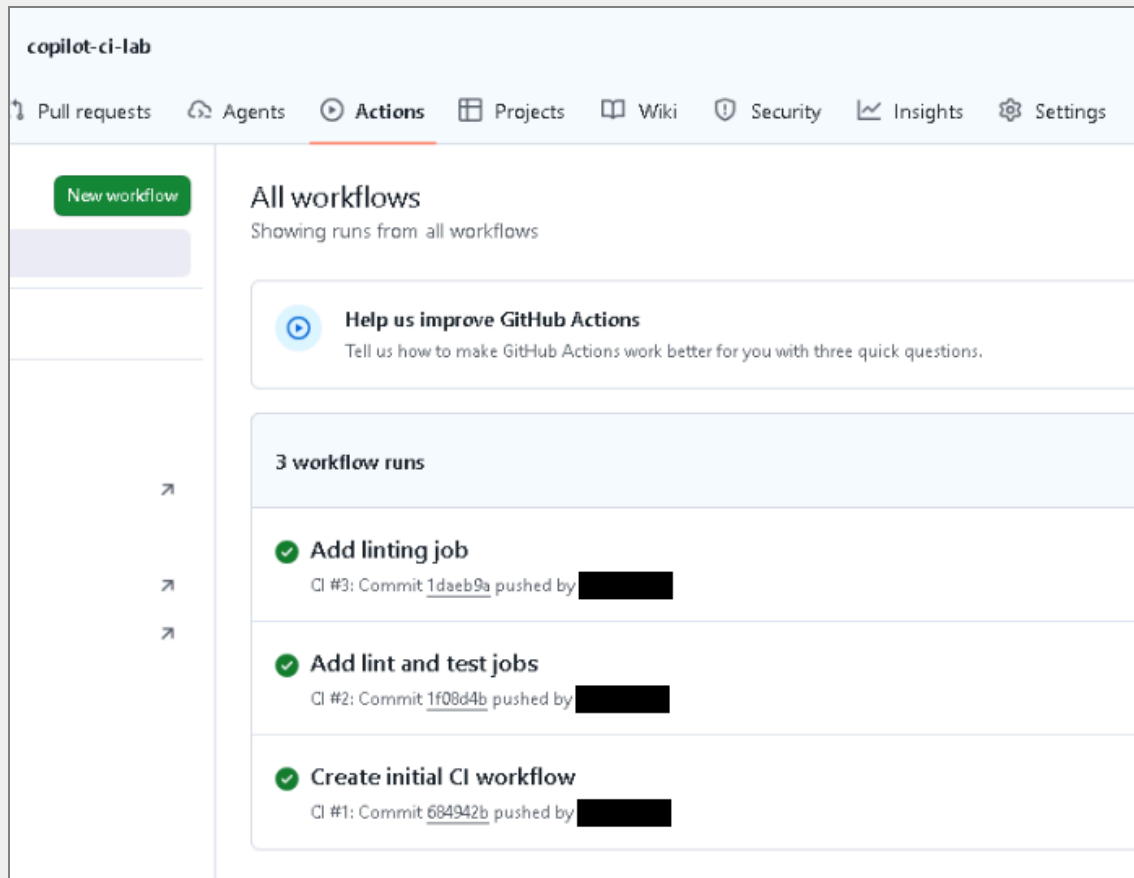
- In **VS Code**, open the **Source Control** view.
- Verify that **main.yml** is listed as a changed file.
- In the **Message** box, enter **Add linting job**.



- Select **Commit**, and then select **Yes**.
 - If required, push your changes by selecting **Sync Changes** or **Push**.
- Verify the job execution order to confirm the **lint** job runs before the **test** job.

💡 Expand this hint for guidance on verifying the job execution order.

- Switch to your browser and navigate to your **copilot-ci-lab** repository on GitHub.
- Select the **Actions** tab.



The screenshot shows the GitHub Actions interface for the repository 'copilot-ci-lab'. The 'Actions' tab is active, displaying 'All workflows' and 'Showing runs from all workflows'. A sidebar on the left includes a 'New workflow' button and a list of workflow runs. The main content area shows a list of workflow runs, each with a green checkmark indicating success. The runs are:

- Add linting job**: CI #3: Commit [1daeb9a](#) pushed by [redacted]
- Add lint and test jobs**: CI #2: Commit [1f08d4b](#) pushed by [redacted]
- Create initial CI workflow**: CI #1: Commit [684942b](#) pushed by [redacted]

- Select the latest workflow run.
- Observe the visualization to confirm the **lint** job runs before the **test** job.





Want to learn more? Review the documentation on [viewing workflow run history](#).

Check your work

Verify

Implement dependency caching and a build matrix

Hints Enabled

No Yes

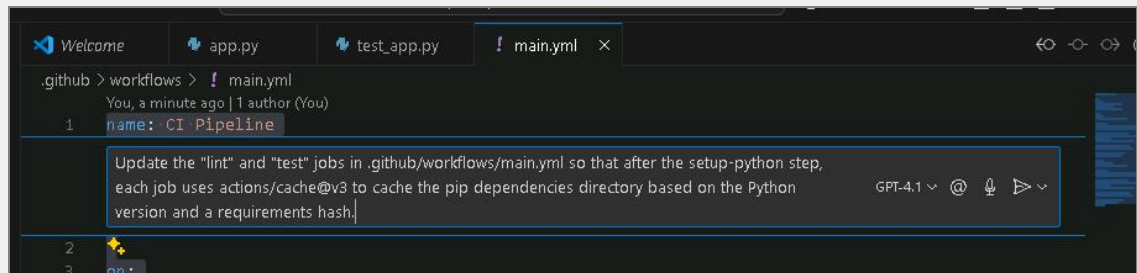
- Add a step to cache pip dependencies using the **actions/cache** action by using the following prompt:



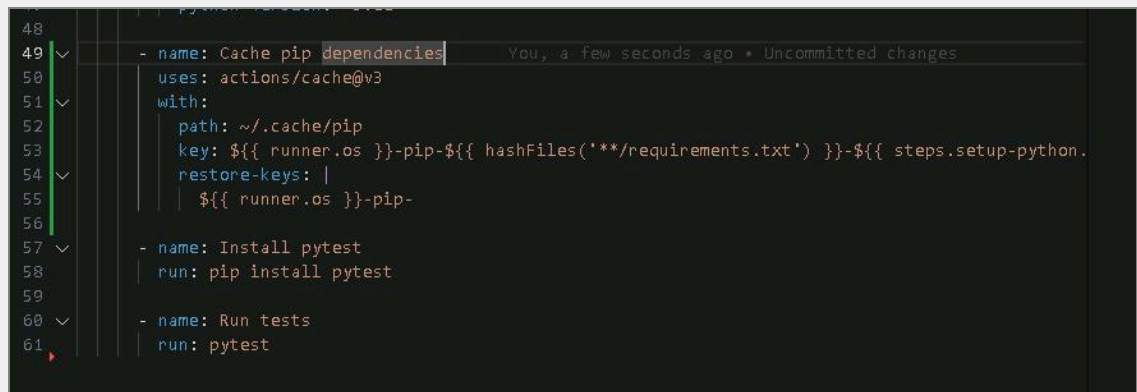
Update the "lint" and "test" jobs in `.github/workflows/main.yml` so that after the `setup-python` step, each job uses `actions/cache@v3` to cache the pip dependencies directory based on the Python version and a requirements hash.

💡 Expand this hint for guidance on adding a step to cache pip dependencies using the actions/cache action. ^

- In **VS Code**, open the **main.yml** file and select the entire script.
- In the **GitHub Copilot Inline Chat**, enter the above supplied prompt.



- Review the suggested changes. Each job should now include a step similar to:



- Adjust the cache key if necessary to match the dependencies in your project.
- Apply the changes to **main.yml** and save the file.

💡 Want to learn more? Review the documentation on [caching dependencies to speed up workflows](#).

⚠️ Caching is essential for speeding up CI workflows. ^

It prevents the need to download and install dependencies from scratch on every run, saving time and network bandwidth.

- Modify the **test** job to use a build matrix for Python versions **3.9** and **3.10** by using the following prompt:

📄 Update the "test" job in `.github/workflows/main.yml` to use a strategy matrix for python-version with values `["3.9", "3.10"]`.

💡 Expand this hint for guidance on modifying the test job to use a build matrix for Python versions 3.9 and 3.10. ^

- In **VS Code**, in the **main.yml** editor, locate the **test** job, and select this section of the script.
- In **GitHub Copilot Inline Chat**, enter the above supplied prompt.

```

35
36 test:
37   needs: lint
38   runs-on: ubuntu-latest
39

```

- Review Copilot's proposed changes. The **test** job should now contain a **strategy/matrix** block similar to:

```

runs-on: ubuntu-latest
strategy:
  matrix:
    python-version: [3.9, 3.10]
  steps:
    - name: Checkout repository
      uses: actions/checkout@v4
    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: ${ matrix.python-version }
    - name: Cache pip dependencies
      uses: actions/cache@v3
      with:
        path: ~/.cache/pip
        key: ${ runner.os }-pip-${ hashFiles('**/requirements.txt') }-${ matrix.python-version }
        restore-keys: |
          ${ runner.os }-pip-

```

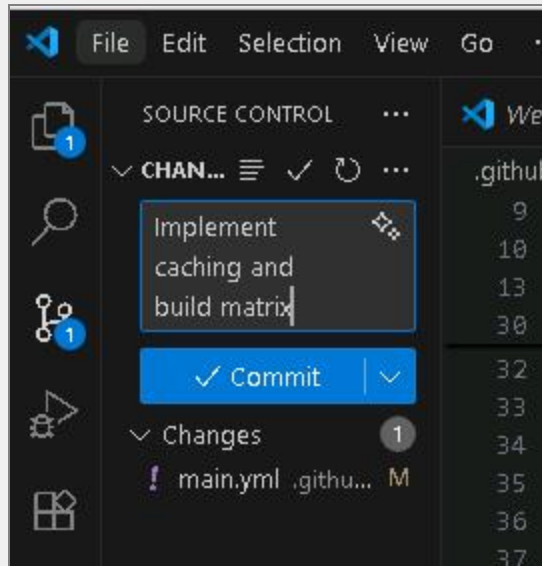
- Apply the changes to the **main.yml** file and save your changes.

💡 Want to learn more? Review the documentation on [using a matrix for your jobs](#).

- Commit the changes by entering **T** Implement caching and build matrix in the **Commit** message textbox.

💡 Expand this hint for guidance on committing the changes.

- In **VS Code**, open the **Source Control** view.
- Verify that **main.yml** is listed as a changed file.
- In the **Message** box, enter **Implement caching and build matrix**.



- Select **Commit** (or **Commit & Push**).
- If necessary, select **Sync Changes** or **Push** to push your commit to GitHub.

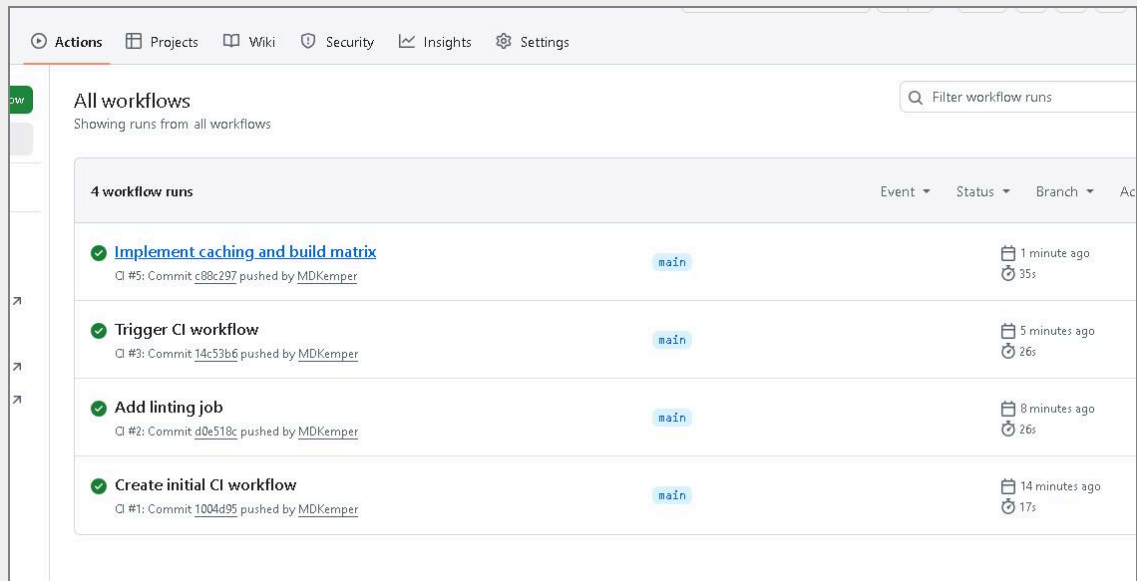
💡 Want to learn more? Review the documentation on [monitoring and troubleshooting workflows](#).

- Confirm the matrix execution by reviewing the **Actions** tab in your **copilot-ci-lab** repository on GitHub.

💡 Expand this hint for guidance on confirming the matrix execution.

- In your browser, open your **copilot-ci-lab** repository on GitHub.
- Select the **Actions** tab.

Note: Your commits may not match these exactly.



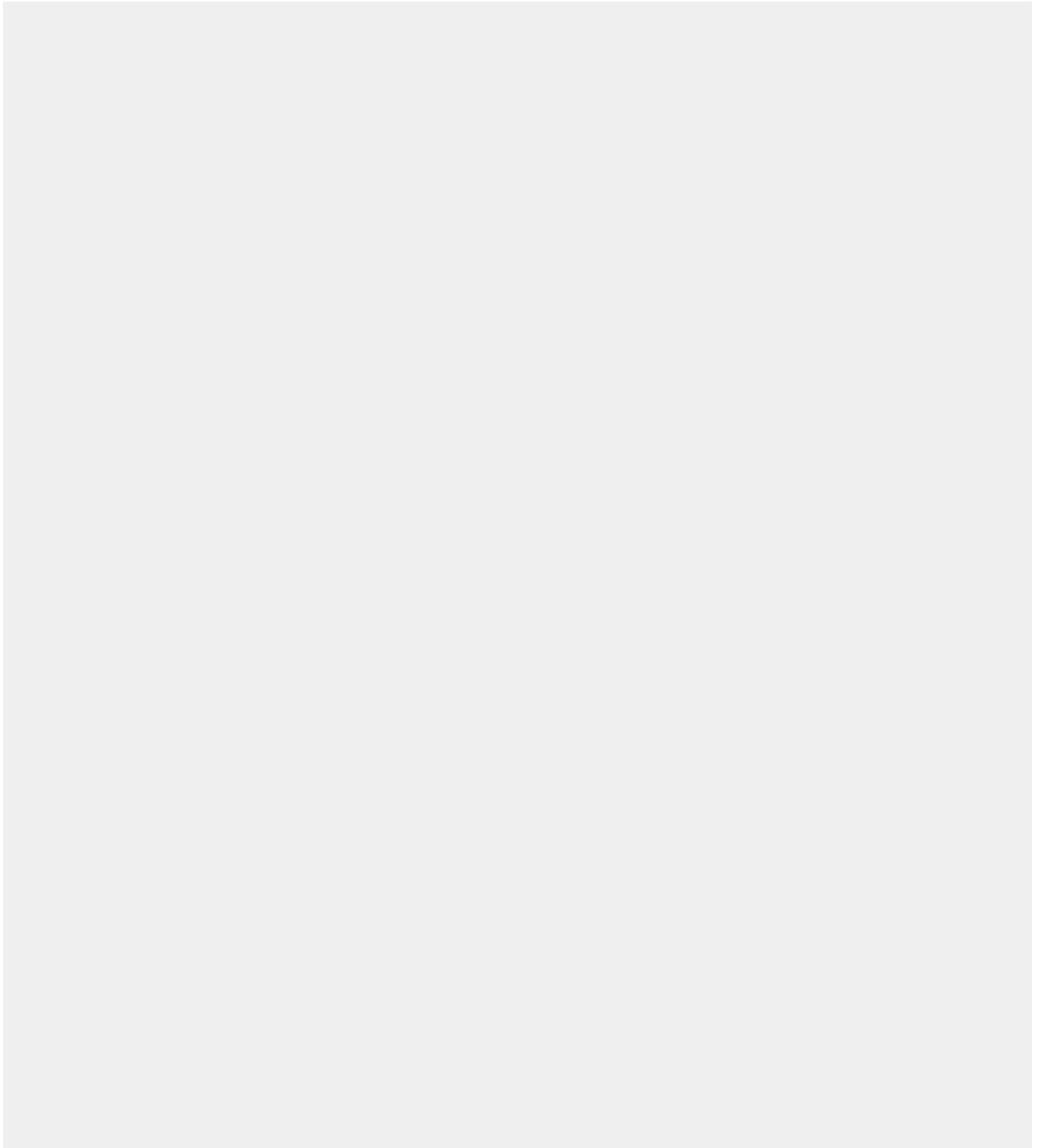
- Select the latest workflow run.
- Confirm that the **test** job has expanded into two separate jobs, one for Python 3.9 and one for Python 3.10.



Check your work



Verify



Summary

Congratulations, you have completed the **Automate CI Workflows by Using GitHub Actions** Challenge Lab.

You have accomplished the following:

- Created a GitHub repository and initial CI workflow.
- Enhanced the CI workflow with a linting job.
- Implemented dependency caching and a build matrix.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

 Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to [Challenge Labs feedback](#).

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Create a Class from a Prompt Using GitHub Copilot [Guided]
- Build Features by Using Copilot in Python [Guided]
- Write Unit Tests Automatically by Using Copilot [Guided]
- Add Comments and Docstrings by Using Copilot [Guided]
- Develop an API Client with GitHub Copilot [Guided]
- Refactor Python Code by Using Copilot [Guided]

- Create Shell Scripts by Using Copilot [Guided]
- Automate CI Workflows by Using GitHub Actions [Guided]
- Document Projects by Using Markdown and Copilot [Guided]
- Refactor and Optimize Code with GitHub Copilot [Guided]
- Build Command-Line Tools by Using Copilot [Guided]
- Utilize Copilot for Python Code Correction [Guided]

